HARAMAYA UNIVERSITY | DEPARTMENT OF COMPUTER SCIENCE

CoSc3053: 90-Day Java Mastery Challenge — Day 5

Challenge Worksheet: Classes, Objects, Encapsulation & BankAccount Validation

Student Name:		**Student ID:**	
Section / Group:		**Date:**	
IDE / JDK Ver:		**Evaluation Grade:**	**/ 10.0 Marks**

1. Challenge Overview & Architectural Learning Objectives

Encapsulation is the fundamental object-oriented principle that binds data (fields) and code (methods) together into a single unit, shielding an object's internal state from unauthorized or invalid external modifications. In this challenge, you will construct a secure, robust **BankAccount** domain model that enforces strict state validation invariants on all financial transactions (deposits, withdrawals, and account transfers).

Concept / Mechanism	Architectural Requirement & Invariant Rule
Encapsulated State	Private fields (accountNumber, accountHolderName, balance, minimumBalance) preventing direct external mutation.
Defensive Validation	Constructor & Mutators reject invalid data (null names, negative deposits, overdrafts violating min balance).
Business Logic Invariants	withdraw() enforces balance >= minimumBalance; deposit() requires amount > 0.0; transferTo() executes atomically.
Demonstrative Test Harness	BankAccountDemo tests successful transactions and verifies graceful handling of invalid operational attempts.

2. Student Guided Challenge: Incomplete Starter Code

Below is the incomplete starter template for **BankAccount.java** and **BankAccountDemo.java**. Your challenge is to complete all **// TODO** blocks, implementing strict validation rules and ensuring your program compiles and executes flawlessly.

```
package edu.haramaya.cosc3053.day5;

/**
 * INCOMPLETE STARTER CODE – DAY 5 OOP CHALLENGE
 * Complete all // TODO sections to implement a fully encapsulated, validated
BankAccount.
 */
public class BankAccount {

    // TODO 1: Declare private encapsulated instance fields
    // - accountNumber (String, immutable/read-only)
    // - accountHolderName (String)
    // - balance (double)
    // - MINIMUM_BALANCE (static final double = 25.00)

    // TODO 2: Implement Multi-Argument Constructor
    public BankAccount(String accountNumber, String accountHolderName, double
initialDeposit) {
        // Validate accountNumber is not null or blank
        // Validate accountHolderName is not null or blank
        // Validate initialDeposit >= MINIMUM_BALANCE; throw IllegalArgumentException
if invalid
        // Initialize instance fields
    }

    // TODO 3: Implement deposit(double amount)
    public boolean deposit(double amount) {
        // Rule: Amount must be strictly greater than 0.0
        // If valid: add to balance, log success message, and return true
        // If invalid: log error message and return false
        return false; // Replace stub
    }

    // TODO 4: Implement withdraw(double amount)
    public boolean withdraw(double amount) {
        // Rule 1: Amount must be strictly greater than 0.0
        // Rule 2: (balance - amount) must be >= MINIMUM_BALANCE ($25.00)
        // If valid: subtract from balance, log success message, and return true
        // If invalid: log descriptive error message and return false
        return false; // Replace stub
    }

    // TODO 5: Implement transferTo(BankAccount targetAccount, double amount)
    public boolean transferTo(BankAccount targetAccount, double amount) {
```

```
        // Rule 1: targetAccount must not be null
        // Rule 2: Attempt withdrawal from THIS account. If successful, deposit into
targetAccount.
        // Return true if transfer succeeds, false otherwise.
        return false; // Replace stub
    }

    // TODO 6: Implement Getter Accessors (No Setters for balance or accountNumber!)
    public String getAccountNumber() { return null; }
    public String getAccountHolderName() { return null; }
    public double getBalance() { return 0.0; }

    // TODO 7: Implement displayAccountSummary()
    public void displayAccountSummary() {
        // Print formatted summary: Account Number, Holder Name, Current Balance
    }
}
```

Incomplete Main Demo Harness (BankAccountDemo.java):

```
package edu.haramaya.cosc3053.day5;

public class BankAccountDemo {
    public static void main(String[] args) {
        System.out.println("========================================");
        System.out.println("   HARAMAYA BANK: OOP ENCAPSULATION DEMO ");
        System.out.println("========================================\n");

        // TODO 8: Instantiate two BankAccount objects with valid initial deposits
        // Account 1: "ACC-1001", "Abebe Bekele", $500.00
        // Account 2: "ACC-1002", "Kebede Lemma", $200.00

        // TODO 9: Test Valid Operations
        // - Deposit $150.00 into Account 1
        // - Withdraw $100.00 from Account 1
        // - Transfer $75.00 from Account 1 to Account 2

        // TODO 10: Test Invalid Operations & Boundary Exceptions
        // - Attempt creating an account with $10.00 initial deposit (Below $25.00
min)
        // - Attempt depositing -$50.00 into Account 1
        // - Attempt withdrawing $1000.00 from Account 2 (Exceeds balance)
        // - Attempt withdrawing $160.00 from Account 2 (Leaves balance below $25.00
min)

        // TODO 11: Print Final Account Summaries for both accounts
    }
}
```

3. Complete Master Reference Solution

Below is the complete, production-grade reference implementation for **BankAccount.java** and **BankAccountDemo.java**. Use this solution to verify your implementation once you have completed the challenge.

```
package edu.haramaya.cosc3053.day5;

/**
 * MASTER REFERENCE SOLUTION — DAY 5 OOP CHALLENGE
 * Demonstrates complete encapsulation, state validation, and atomic transfers.
 */
public class BankAccount {

    // 1. Encapsulated Private Instance Fields
    private final String accountNumber;      // Read-only immutable identifier
    private String accountHolderName;
    private double balance;

    // Class Invariant Constant
    public static final double MINIMUM_BALANCE = 25.00;

    // 2. Multi-Argument Constructor with Defensive Validation
    public BankAccount(String accountNumber, String accountHolderName, double
initialDeposit) {
        if (accountNumber == null || accountNumber.isBlank()) {
            throw new IllegalArgumentException("Account number cannot be null or
blank.");
        }
        if (accountHolderName == null || accountHolderName.isBlank()) {
            throw new IllegalArgumentException("Account holder name cannot be null or
blank.");
        }
        if (initialDeposit < MINIMUM_BALANCE) {
            throw new IllegalArgumentException(String.format(
                "Initial deposit ($%.2f) is below the minimum required balance
($%.2f).",
                initialDeposit, MINIMUM_BALANCE));
        }

        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = initialDeposit;
    }

    // 3. Deposit Operation with Positive Amount Validation
    public boolean deposit(double amount) {
        if (amount <= 0.0) {
            System.out.printf("❌ [REJECTED] Deposit failed for %s: Amount ($%.2f)
must be positive.%n",
```

```
                accountNumber, amount);
            return false;
        }
        this.balance += amount;
        System.out.printf("☑ [SUCCESS] Deposited $%.2f into %s. New Balance:
$%.2f%n",
            amount, accountNumber, this.balance);
        return true;
    }

    // 4. Withdrawal Operation with Overdraft & Minimum Balance Invariant Guard
    public boolean withdraw(double amount) {
        if (amount <= 0.0) {
            System.out.printf("❌ [REJECTED] Withdrawal failed for %s: Amount ($%.2f)
must be positive.%n",
                accountNumber, amount);
            return false;
        }
        if ((this.balance - amount) < MINIMUM_BALANCE) {
            System.out.printf("❌ [REJECTED] Withdrawal of $%.2f from %s denied.
Remaining balance ($%.2f) would violate minimum required balance ($%.2f).%n",
                amount, accountNumber, (this.balance - amount), MINIMUM_BALANCE);
            return false;
        }
        this.balance -= amount;
        System.out.printf("☑ [SUCCESS] Withdrew $%.2f from %s. New Balance:
$%.2f%n",
            amount, accountNumber, this.balance);
        return true;
    }

    // 5. Atomic Account Transfer
    public boolean transferTo(BankAccount targetAccount, double amount) {
        if (targetAccount == null) {
            System.out.printf("❌ [REJECTED] Transfer failed from %s: Target account
is null.%n", accountNumber);
            return false;
        }
        System.out.printf("🔄 [INITIATING] Transferring $%.2f from %s to %s...%n",
            amount, this.accountNumber, targetAccount.getAccountNumber());

        if (this.withdraw(amount)) {
            targetAccount.deposit(amount);
            System.out.printf("☑ [TRANSFER COMPLETE] $%.2f successfully transferred
from %s to %s.%n",
                amount, this.accountNumber, targetAccount.getAccountNumber());
            return true;
        } else {
            System.out.printf("❌ [TRANSFER FAILED] Transfer aborted due to
insufficient funds in %s.%n", accountNumber);
```

```
            return false;
        }
    }

    // 6. Controlled Accessors
    public String getAccountNumber() { return accountNumber; }
    public String getAccountHolderName() { return accountHolderName; }
    public double getBalance() { return balance; }

    public void setAccountHolderName(String accountHolderName) {
        if (accountHolderName != null && !accountHolderName.isBlank()) {
            this.accountHolderName = accountHolderName;
        }
    }

    // 7. Display Summary
    public void displayAccountSummary() {
        System.out.printf("  Account No: %-10s | Holder: %-15s | Balance: $%.2f%n",
            accountNumber, accountHolderName, balance);
    }
}
```

Full Master Test Harness (BankAccountDemo.java):

```
package edu.haramaya.cosc3053.day5;

public class BankAccountDemo {
    public static void main(String[] args) {
        System.out.println("========================================");
        System.out.println("   HARAMAYA BANK: OOP ENCAPSULATION DEMO ");
        System.out.println("========================================\n");

        // 1. Valid Account Instantiations
        System.out.println("--- 1. CREATING VALID ACCOUNTS ---");
        BankAccount acc1 = new BankAccount("ACC-1001", "Abebe Bekele", 500.00);
        BankAccount acc2 = new BankAccount("ACC-1002", "Kebede Lemma", 200.00);
        acc1.displayAccountSummary();
        acc2.displayAccountSummary();

        // 2. Valid Financial Transactions
        System.out.println("\n--- 2. EXECUTING VALID TRANSACTIONS ---");
        acc1.deposit(150.00);
        acc1.withdraw(100.00);
        acc1.transferTo(acc2, 75.00);

        // 3. Testing Invalid Operations & Guards
        System.out.println("\n--- 3. TESTING EXCEPTION & BOUNDARY GUARDS ---");

        // Constructor Guard Test
```

```
        try {
            System.out.println("Attempting to open account with $10.00 deposit (Below
$25.00 min):");
            BankAccount badAcc = new BankAccount("ACC-1003", "Invalid User", 10.00);
        } catch (IllegalArgumentException e) {
            System.out.println("  Caught Expected Exception: " + e.getMessage());
        }

        // Negative Deposit Test
        System.out.println("\nAttempting negative deposit (-$50.00):");
        acc1.deposit(-50.00);

        // Overdraft / Minimum Balance Violation Test
        System.out.println("\nAttempting withdrawal ($260.00) leaving balance below
$25.00 min:");
        acc2.withdraw(260.00);

        // 4. Final Account Summaries
        System.out.println("\n========================================");
        System.out.println("          FINAL ACCOUNT SUMMARIES        ");
        System.out.println("========================================");
        acc1.displayAccountSummary();
        acc2.displayAccountSummary();
        System.out.println("========================================");
    }
}
```

4. Executed Visual Console Snapshots

When your solution is correctly implemented and executed, your console output must match the visual execution trace below:

SNAPSHOT 4.1: VALID TRANSACTION & TRANSFER EXECUTION TRACE

```
========================================
   HARAMAYA BANK: OOP ENCAPSULATION DEMO
========================================

--- 1. CREATING VALID ACCOUNTS ---
  Account No: ACC-1001   | Holder: Abebe Bekele    | Balance: $500.00
  Account No: ACC-1002   | Holder: Kebede Lemma    | Balance: $200.00

--- 2. EXECUTING VALID TRANSACTIONS ---
☑ [SUCCESS] Deposited $150.00 into ACC-1001. New Balance: $650.00
☑ [SUCCESS] Withdrew $100.00 from ACC-1001. New Balance: $550.00
🔄 [INITIATING] Transferring $75.00 from ACC-1001 to ACC-1002...
☑ [SUCCESS] Withdrew $75.00 from ACC-1001. New Balance: $475.00
☑ [SUCCESS] Deposited $75.00 into ACC-1002. New Balance: $275.00
☑ [TRANSFER COMPLETE] $75.00 successfully transferred from ACC-1001 to ACC-1002.
```

SNAPSHOT 4.2: EXCEPTION & BOUNDARY GUARD REJECTION TRACE

```
--- 3. TESTING EXCEPTION & BOUNDARY GUARDS ---
Attempting to open account with $10.00 deposit (Below $25.00 min):
  Caught Expected Exception: Initial deposit ($10.00) is below the minimum required
balance ($25.00).

Attempting negative deposit (-$50.00):
❌ [REJECTED] Deposit failed for ACC-1001: Amount (-$50.00) must be positive.

Attempting withdrawal ($260.00) leaving balance below $25.00 min:
❌ [REJECTED] Withdrawal of $260.00 from ACC-1002 denied. Remaining balance ($15.00) would
violate minimum required balance ($25.00).

========================================
          FINAL ACCOUNT SUMMARIES
========================================
  Account No: ACC-1001   | Holder: Abebe Bekele    | Balance: $475.00
  Account No: ACC-1002   | Holder: Kebede Lemma    | Balance: $275.00
========================================
```

5. Self-Guided Architectural Reflection Checkpoints

Answer the following deep architectural questions to consolidate your understanding of Object-Oriented Principles:

Checkpoint 5.1 (Direct Field Exposure Hazard):

If the 'balance' field were declared as 'public double balance', explain how a malicious or buggy external class could corrupt the BankAccount object's state. What invariant would be broken?

Student Response / Analysis:

Checkpoint 5.2 (Controlled Mutators vs. Direct Setters):

Why is providing a 'public void setBalance(double b)' method almost as dangerous as making balance public? Why are deposit() and withdraw() superior mutator abstractions?

Student Response / Analysis:

Checkpoint 5.3 (Immutability Benefits):

Why is 'accountNumber' declared as 'private final String'? What software engineering benefits arise from making unique entity identifiers immutable after construction?

Student Response / Analysis:

Checkpoint 5.4 (Atomic Transfer Analysis):

In transferTo(), why is calling 'this.withdraw(amount)' before 'targetAccount.deposit(amount)' critical? What would happen if deposit() were called first and withdraw() subsequently failed?

Student Response / Analysis:

6. Evaluation Rubric (/10.0 Marks)

Evaluation Dimension	Performance Criteria & Architectural Expectation	Max Marks
Encapsulation & Scope	All instance fields private; balance protected from direct external access.	2.0 Marks
Defensive Validation	Constructor and methods correctly reject invalid amounts & enforce $25.00 min balance.	2.5 Marks
Transfer Atomicity	transferTo() executes cleanly using withdraw/deposit composition without state corruption.	2.0 Marks
Reflection Checkpoints	Deep, accurate answers to all 4 architectural checkpoints demonstrating OOP mastery.	2.5 Marks
Lab Defense & Code	Code compiles cleanly; student successfully explains state protection during defense.	1.0 Mark

Signature: _____ Date: _____ Final Score: ____ / 10.0